DVSA Vulnerability

Discovery & Remediation

ICS-344: Information Security | KFUPM | Term 252

Team Members: Lujain Aljehani • Meznah Alshubaian • Lamees Alharbi

DVSA URL: http://dvsa-website-577211135850-us-east-1.s3-website.us-east-1.amazonaws.com

Project Overview

What is DVSA?

Damn Vulnerable Serverless Application, a deliberately insecure app deployed on AWS for security training.

Our Goal

Identify, exploit, fix, and verify 10 official vulnerabilities in a real AWS serverless environment.

Architecture

S3 frontend → API Gateway → Lambda → DynamoDB. Vulnerabilities exist at every layer of this pipeline.

AWS Services Used:

- API Gateway
- Lambda
- DynamoDB
- S3
- Cognito
- IAM
- CloudWatch

10 Vulnerabilities

01	Event Injection	Lamees	FIXED
02	Broken Authentication	Meznah	FIXED
03	Sensitive Data Exposure	Lamees	FIXED
04	Insecure Cloud Config	Meznah	FIXED
05	Broken Access Control	Meznah	FIXED
06	Denial of Service	Lujain	FIXED
07	Over-Privileged Functions	Lujain	FIXED
08	Logic Vulnerabilities	Lujain	FIXED
09	Vulnerable Dependencies	Lamees	FIXED
10	Unhandled Exceptions	Lujain	FIXED

01

Event Injection & Vulnerable Dependencies

Affected: DVSA-ORDER-MANAGER Lambda / node-serialize package

Exploit

Sent a malicious payload using the _$$ND_FUNC$$_ marker through the /order API. The node-serialize library evaluated and executed the injected JavaScript function during deserialization, before any application logic ran.
Result: CloudWatch logs confirmed LAMEES_LESSON_1_SUCCESS — proving arbitrary code executed on the Lambda backend.

```
2026-04-25T21:38:09.310Z    2026-04-25T21:38:09.310Z eb0f2555-5620-48ee-b949-33dab392d0e4 ERROR LAMEES_LESSON_1_SUCCESS
2026-04-25T21:38:09.310Z    eb0f2555-5620-48ee-b949-33dab392d0e4    ERROR    LAMEES_LESSON_1_SUCCESS
```

Fix Applied

Replaced serialize.unserialize() with JSON.parse() in DVSA-ORDER-MANAGER/order-manager.js for both request body and headers. JSON.parse() only parses data and never evaluates functions, completely eliminating the injection vector.

```
var req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;
var headers = typeof event.headers === "string" ? JSON.parse(event.headers) : event.headers;
```

Verified

After the fix, the same injection payload was sent again. CloudWatch logs showed only START, END, and REPORT entries with no LAMEES_LESSON_1_SUCCESS message. The injected function no longer executes.

```
2026-04-25T21:42:07.687Z    START RequestId: f61b140e-01a0-4777-938b-99061644b5de Version: $LATEST
START RequestId: f61b140e-01a0-4777-938b-99061644b5de Version: $LATEST
2026-04-25T21:42:09.093Z    END RequestId: f61b140e-01a0-4777-938b-99061644b5de
END RequestId: f61b140e-01a0-4777-938b-99061644b5de
```

02 Broken Authentication

Affected: DVSA-ORDER-MANAGER Lambda / JWT handling

Exploit

Captured a valid JWT for User B. Decoded the payload, replaced username and sub fields with User C's identity, re-encoded it with the original signature (B).

Result: The backend accepted the forged token and returned User C's order details.

```
meznah@meznah-VirtualBox:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
  "status": "ok",
  "order": {
    "address": {
      "name": "Meznah Yahya",
      "address": "Khobar",
      "phone": [redacted],
      "email": ""
    },
    "confirmationToken": [redacted],
    "itemList": {
      "9012": 1
    },
    "orderId": [redacted],
    "orderStatus": 200,
    "payentTS": 1776878122,
    "totalAmount": 32,
    "userId": [redacted]
  }
}
```

Fix Applied

Added JWT signature verification using trusted public keys.
The backend now verifies the signature and validates issuer and expiry before trusting any claim.

```
var jwt = auth_header.replace(/^Bearer\s+/i, "").trim();
if (!jwt) {
  return callback(null, resp(401, { status: "err", msg: "missing authorization" }));
}

verifyCognitoJwt(jwt).then((claims) => {
    var user = claims.username || claims["cognito:username"] || claims.sub;
    if (!user) {
        return callback(null, resp(401, { status: "err", msg: "missing subject" }));
    }

    var isAdmin = false;
```

```
    }
}).catch((e) => {
    console.log("JWT verify failed:", e);
    return callback(null, resp(401, { status: "err", msg: "invalid token" }));
});
};
```

Verified

The forged token now returns an error (invalid token).
A legitimate token still works normally, but User B can no longer access User C's orders.

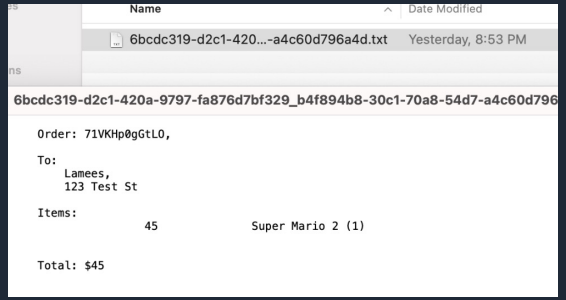
```
meznah@meznah-VirtualBox:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
{
  "status": "err",
  "msg": "invalid token"
}
```

03 Sensitive Information Disclosure

Affected: DVSA-ADMIN-GET-RECEIPT Lambda / S3 signed URLs

Exploit

As a regular non-admin user, called the get-receipt action on the /order API with a valid order ID, year, month, and day. The DVSA-ADMIN-GET-RECEIPT function was invoked with no authorization check. Result: Received a pre-signed S3 URL and downloaded a zip file containing a real receipt — customer name (Lamees), address (123 Test St), item (Super Mario 2), total ($45).



Fix Applied

Added if (!isAdmin) authorization check in the get-receipt case of DVSA-ORDER-MANAGER/order-manager.js. If the user is not an admin, the function returns 403 Forbidden immediately without ever invoking DVSA-ADMIN-GET-RECEIPT.

```
case "get-receipt":
    if (!isAdmin) {
        const response = {
            statusCode: 403,
            headers: corsHeaders,
            body: JSON.stringify({ "status": "err", "msg": "Forbidden: admin only" })
        };
        callback(null, response);
        return;
    }
    payload = {
        "user": user,
        "order-id": req["order-id"],
        "isAdmin": isAdmin,
        "year": req["year"],
        "month": req["month"],
        "day": req["day"]
    };
    functionName = "DVSA-ADMIN-GET-RECEIPT";
    break;
```

Verified

Non-admin users can no longer generate receipt URLs for other users' orders. Unauthorized requests now return a 403 Forbidden response.

```
~ $ curl -s "$API"   -H "content-type: application/json"   -H "authorization: $TOKEN"
th":"04","day":"25"}' | python3 -m json.tool
{
    "status": "err",
    "msg": "Forbidden: admin only"
}
```

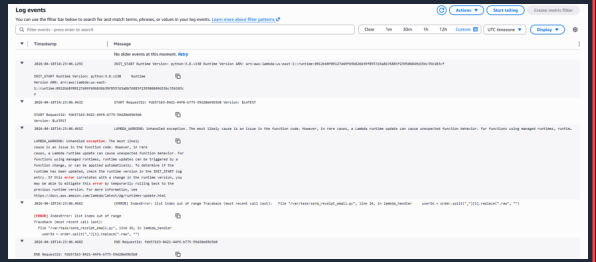
04 Insecure Cloud Configuration

Affected: S3 bucket / Lambda function (DVSA-SEND-RECEIPT-EMAIL)

Exploit

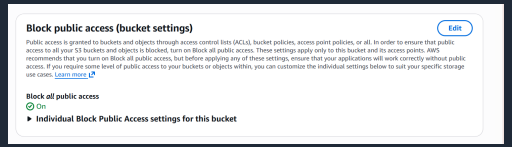
Uploaded a malicious file directly to the DVSA S3 bucket due to weak bucket access policies. The file was processed by a Lambda function.

Result: Attacker ‘s file content was executed and processed by the backend Lambda.



Fix Applied

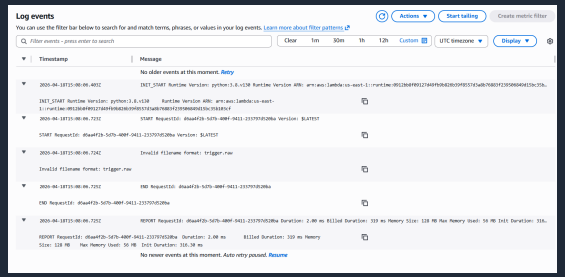
Enabled block public access in S3. Added filename validation (ends with .raw and contains _). Replaces unsafe execution with safe file handling.




```
order = key.split("/",1)[-1]
orderId = order.split("_")[0]
file_name= order.rsplit("/",1)[-1]
if not file_name.endswith('.raw') or "_" not in file_name:
    print("Invalid filename format:", file_name)
    return {
        "statusCode": 400,
        "body": "invalid file name format"
    }

file_name_parts= file_name.split("_")
userId= file_name_parts[1].replace(".raw", "")

s3 = boto3.client('s3')
# download file to /tmp

download_path = f'/tmp/{str(uuid.uuid4())}.txt'
date = datetime.datetime.now().strftime('%Y-%m-%d %H:%M')
with open(download_path, 'a') as f:
    f.write(f"\n--------------------\nDate: {date}\n")
```

Verified

Attempts to upload malicious files to the bucket are now denied. The Lambda function no longer processes unvalidated attacker-controlled files.



05 Broken Access Control

Affected: DVSA-ADMIN-UPDATE-ORDERS Lambda function

Exploit

The DVSA-ADMIN-UPDATE-ORDERS function was accessed through a normal user request using the API. The order status was changed to “paid” without completing the payment process.

Result: Order appeared as paid without any actual payment.

```
meznah@meznah-VirtualBox:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}"
{"status":"ok","order":{"address":{"name":"Meznah Yahya","address":"Khobar","
phone":"222222","email":""},"confirmationToken":" ","itemList":{"1020":1},"or
derId":"b79ea1fd-d74e-4705-bc85-f84d6c989309","orderStatus":100,"paymentTS":1
776866728,"totalAmount":0,"[redacted]"}}me
znah@meznah-VirtualBox:~$ cucurl -s -X POST "$API" \
```

```
{"status":"err","msg":"unknown action"}meznah@meznah-VirtualBox:~$ curl -s "$
API" \                          curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}"
{"status":"ok","order":{"address":{"name":"Hacker","email":"hacker@test.com",
"address":"123 Hack St"},"confirmationToken":"FREE_RIDE_TOKEN","itemList":{"1
020":1},"orderId":"b79ea1fd-d74e-4705-bc85-f84d6c989309","orderStatus":120,"p
aymentTS":1711713536,"totalAmount":0,"userId":"a448d4c8-e0c1-70b4-4cb1-940ee8
```

Fix Applied

The DVSA-ADMIN-UPDATE-ORDERS function is now protected by role checks.
Order status updates are only allowed for authorized users.
Requests from public functions are blocked from performing admin operations.

```python
cognito = boto3.client("cognito-idp")
user_pool_id = os.environ["userpoolid"]

try:
    user_data = cognito.admin_get_user(
        UserPoolId=user_pool_id,
        Username=user
    )
except ClientError:
    return {"status": "err", "msg": "Could not verify user"}

is_admin = "false"
for attr in user_data.get("UserAttributes", []):
    if attr["Name"] == "custom:is_admin":
        is_admin = attr["Value"]
        break

if is_admin != "true":
    return {"status": "err", "msg": "forbidden"}
```

Verified

Regular users can no longer call the admin update function.
Attempting to change order status without payment now returns an error (unknown action).

```
meznah@meznah-VirtualBox:~$ curl -s -X POST "$API" \
-H 'Content-Type: application/json' \
-H "Authorization: $TOKEN" \
--data @lesson5_payload.json
{"status":"err","msg":"unknown action"}meznah@meznah-VirtualBox:~$ cur
API" \                          curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}"
{"status":"ok","order":{"address":{"name":"Meznah Yahya","address":"Kh
phone":"222222","email":""},"confirmationToken":" ","itemList":{"1014"
derId":"[redacted]","orderStatus":100,"payme
```

06 Denial of Service

Affected: DVSA-ORDER-BILLING Lambda / API Gateway

Exploit

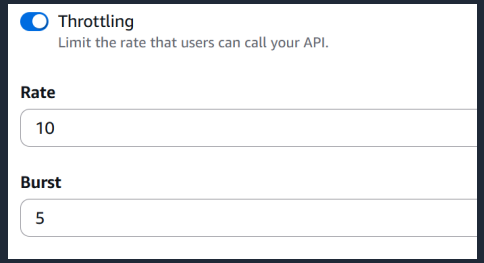
Sent 50 concurrent billing requests simultaneously using curl with & operator. The billing Lambda was overwhelmed and crashed.

Result: Mass 500 Internal Server Errors. Legitimate users could not complete checkout.

```
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
```

Fix Applied

configured API Gateway dvsa stage throttling: Rate 5, Burst 10.



Verified

Running the same 50 concurrent requests now produces 429 Too Many Requests responses, confirming the rate limit is blocking excessive requests.

```
{"message":"Too Many Requests"}{"message":"Too Many Requests"}

{"message":"Too Many Requests"}{"message":"Too Many Requests"}

{"message":"Too Many Requests"}
{"message":"Too Many Requests"}
{"message":"Too Many Requests"}{"message":"Too Many Requests"}

{"message":"Too Many Requests"}{"message":"Too Many Requests"}

{"message":"Too Many Requests"}
{"message":"Too Many Requests"}
{"message":"Too Many Requests"}{"message":"Too Many Requests"}
{"message":"Too Many Requests"}
{"message":"Too Many Requests"}
```

07 Over-Privileged Functions

Affected: DVSA-SEND-RECEIPT-EMAIL Lambda / IAM Role

Exploit

Reviewed the role of DVSA-SEND-RECEIPT-EMAIL. Found policies that apply to all resources (using '*').

IAM Policy Simulator confirmed:
S3 GetObject/PutObject Allowed on ANY bucket.
DynamoDB Scan/Delete Allowed on ANY table.

Its beyond what a receipt email needs.

Amazon DynamoDB	Scan	allowed
Amazon DynamoDB	GetItem	allowed
Amazon DynamoDB	PutItem	allowed
Amazon DynamoDB	DeleteItem	allowed

Amazon S3	PutObject	allowed
Amazon S3	GetObject	allowed

Fix Applied

Restricted SendReceiptFunctionRolePolicy1 to only dvsa-receipts bucket.
Restricted Policy2 to only GetItem on DVSA-ORDERS-DB table.
Replaced Policy3 with specific SES send permissions.
Removed AmazonSESFullAccess managed policy.

Policy editor

```
1 ▾ {
2 ▾   "Statement": [
3 ▾     {
4 ▾       "Action": [
5         "s3:GetObject",
6         "s3:PutObject"
7       ],
8 ▾     "Resource": [
9         "arn:aws:s3:::dvsa-*receipts*/*"
10      ],
11      "Effect": "Allow"
12    }
13  ]
14 }
```

Policy editor

```
1 ▾ {
2 ▾   "Statement": [
3 ▾     {
4 ▾       "Action": [
5         "ses:SendEmail",
6         "ses:SendRawEmail"
7       ],
8       "Resource": "*",
9       "Effect": "Allow"
10    }
11  ]
12 }
```

licy editor

```
{
  "Statement": [
    {
      "Action": [
        "dynamodb:GetItem"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-east-1:577211135850:table/DVSA-ORDERS-DB"
      ],
      "Effect": "Allow"
    }
  ]
}
```

Verified

IAM Policy Simulator now shows Denied for S3 buckets and DynamoDB tables.

The function can only access the specific resources it needs.

	Service	Action	Permission
▸	Amazon S3	GetObject	denied
▸	Amazon S3	PutObject	denied
▸	Amazon DynamoDB	Scan	denied Imp
▸	Amazon DynamoDB	DeleteItem	denied Imp

08 Logic Vulnerabilities (Race Condition)

Affected: DVSA-ORDER-UPDATE Lambda / DynamoDB

Exploit

Sent a billing request and an update request simultaneously.

Billing processed payment for 1 item while update changed quantity to 5.
DynamoDB confirmed: itemList = 5 items, totalAmount = $0.


```
},
"itemList": {
  "M": {
    "1": {
      "N": "5"
    }
  }
},
"orderId": {
  "S": "16f21aa0-5edb-45d8-9fda-51aff5e9e46a"
},
"orderStatus": {
  "N": "100"
},
"paymentTS": {
  "N": "1776022393"
},
"totalAmount": {
  "N": "0"
},
"userId": {
  "S": "54181498-6091-703c-c083-1e71736e0895"
}
```


Fix Applied

Fix in DVSA-ORDER-UPDATE:
1. Status check: rejects update if orderStatus != 100

2. DynamoDB conditional update by checking both status and absence of paymentTS.

```
if response["Item"]["orderStatus"] != 100:
    return {"status": "err", "msg": "order cannot be modified"}


try:
    response = table.update_item(
        Key={"orderId": orderId, "userId": userId},
        UpdateExpression="SET itemList = :itemList",
        ConditionExpression="orderStatus = :open AND attribute_not_exists(paymentTS)",
        ExpressionAttributeValues={':itemList': itemList, ':open': 100}
    )
except Exception as e:
    return {"status": "err", "msg": "Order cannot be modified (payment in progress or already updated)"}
```

Verified

The update request now returns 'Order cannot be modified (payment in progress)'.

DynamoDB conditional update blocks concurrent modifications at the database.

```
~ $ wait{
  "status": "err",
  "msg": "Order cannot be modified (payment in progress or already updated)"
}

{
  "status": "err",
  "msg": "could not process order"
}
```

09 Vulnerable Dependencies

Affected: DVSA-ORDER-MANAGER Lambda / node-serialize

Exploit

The node-serialize package in DVSA-ORDER-MANAGER was used to deserialize attacker-controlled input. Its $$ND_FUNC$$ marker causes it to evaluate JavaScript functions during deserialization, a dangerous capability never intended for untrusted input. Result: A single crafted JSON request caused remote code execution entirely because of an unsafe third-party library.

```
START RequestId: eb0f2555-5620-48ee-b949-33dab392d0e4 Version: $LATEST

2026-04-25T21:38:09.310Z        2026-04-25T21:38:09.310Z eb0f2555-5620-48ee-b949-33dab392d0e4 ERROR LAMEES_LESSON_1_SUCCESS

2026-04-25T21:38:09.310Z        eb0f2555-5620-48ee-b949-33dab392d0e4    ERROR   LAMEES_LESSON_1_SUCCESS
```

Fix Applied

Replaced serialize.unserialize() with JSON.parse() in order-manager.js. JSON.parse() is a built-in safe alternative that never evaluates functions. The node-serialize library is no longer in the request path.

```
var req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;
var headers = typeof event.headers === "string" ? JSON.parse(event.headers) : event.headers;
```

Verified

After the fix, the same injection payload was sent. CloudWatch logs showed only START, END, and REPORT entries — no LAMEES_LESSON_1_SUCCESS message. The vulnerable dependency no longer poses a risk.

```
2026-04-25T21:42:07.687Z        START RequestId: f61b140e-01a0-4777-938b-99061644b5de Version: $LATEST

START RequestId: f61b140e-01a0-4777-938b-99061644b5de Version: $LATEST

2026-04-25T21:42:09.093Z        END RequestId: f61b140e-01a0-4777-938b-99061644b5de

END RequestId: f61b140e-01a0-4777-938b-99061644b5de
```

10 Unhandled Exceptions

Affected: DVSA-ORDER-BILLING and DVSA-ORDER-GET Lambda

Exploit

Sent malformed requests to two endpoints:

- Billing with no real cart total: KeyError: 'total' leaked from order_billing.py line 64
- Get order with wrong type: AttributeError leaked from get_order.py line 32

Internal file names, line numbers, and source code exposed to anyone.

```
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"billing","order-id":"'"$EX
CEPTION_ORDER"'","data":{"ccn":"4242424242424242","exp":"11/25","cvv":"123"}}' | jq
{
  "errorMessage": "'total'",
  "errorType": "KeyError",
  "stackTrace": [
    "  File \"/var/task/order_billing.py\", line 64, in lambda_handler\n    cartTotal = float(res['total'])\n"
  ]
}
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"get","order-id":"16f21aa0-
5edb-45d8-9fda-51aff5e9e46a"}' | jq
{
  "errorMessage": "'bool' object has no attribute 'lower'",
  "errorType": "AttributeError",
  "stackTrace": [
    "  File \"/var/task/get_order.py\", line 32, in lambda_handler\n    is_admin = json.loads(event.get(\"isAdmin\", \"fals
e\").lower())\n"
  ]
```

Fix Applied

Added try/except in order_billing.py around cart total calculation returning only a generic error message.

Fixed get_order.py with isAdmin handling.

All internal errors now logged to CloudWatch only.

```python
try:
    res = json.loads(req.data)
    cartTotal = float(res['total'])
    missings = res.get("missing", {})
except (KeyError, ValueError, Exception) as e:
    print("Billing error (internal):", str(e))
    return {"status": "err", "msg": "could not process order"}
```

```python
is_admin_raw = event.get("isAdmin", "false")
if isinstance(is_admin_raw, bool):
    is_admin = is_admin_raw
else:
    is_admin = json.loads(str(is_admin_raw).lower())
```

Verified

Both endpoints now return generic messages with no internal details.

Before: full stack trace with file paths and code.

After: {status: err, msg: could not process order}.

```
{
  "status": "err",
  "msg": "could not process order"
}
```

Key Takeaways

Verify, Don't Trust

Never trust JWT claims without verifying the signature. Unverified tokens are the same as no authentication.

Least Privilege

Every Lambda function should only have permissions for exactly what it needs.

Lock Critical Operations

Use conditional updates and locking to prevent race conditions in workflows with multiple steps.

Rate Limiting is Essential

Serverless functions have no DoS by default. Always add throttling at the API Gateway and Lambda level.

Never Expose Internals

Catch all exceptions and return only generic messages to users and log full details to CloudWatch.

Audit Dependencies

Third-party packages can introduce critical vulnerabilities. Regularly scan and replace unsafe libraries.

Thank You

Questions?